

Coding the Solution to Ireland's Scam Call Issue



Fionn Campbell & Emma Keane

Project reference number: PR2878

Stand number: 2401

Junior Technology

Moate Community School, Moate, Co. Westmeath.



Table of contents:

Page:	Section:
3-4	Project summary
5	Meet the team
6	Aims and objectives
7-8	Background Research
9-16	Market research survey
17	Design Brief
18-37	Experimental methods
38-41	App features in-action
42	Conclusions
43	Recommendations
44	Acknowledgements
45-46	References
See 'Stand 2401 Appendices'	Appendices

Project Summary:

This project aimed to design an app to alert users if they receive a potentially dangerous call from an unknown number and warn them appropriately.

These calls, commonly called 'scam calls' are a massive issue in Ireland today, and often involve an attempt to obtain money or personal information from the victim. The incidence of these calls has risen by 440% over the course of the pandemic according to an Garda Síochána. This issue was exacerbated by the leak of patient information following the cyber-attack on the HSE in 2021. Scammers use many constantly evolving methods to deceive and attack victims, the most common perhaps being impersonating trustworthy organisations such as banks. As a result, scam calls can be very difficult for users to recognise. Emma's grandmother herself had to cancel her bank card after being tricked into giving its details by a scammer. This was the inspiration for our app 'ScamBlocker'.

To determine the extent of this problem in our community, a market research survey was carried out among 102 students in our school in which respondents provided information on their experience with the issue and whether they would avail of an app to deter such calls. The results showed that 70% of students receive suspicious calls at least weekly, that 80% found this a notable issue and that 91% felt they would benefit from our app.

Following market research, the following necessary features were identified to produce a working prototype of the app to adequately reduce interaction with scam calls. These included the ability to: detect incoming numbers and determine their presence on the user's contacts list, send a notification requesting user feedback on a number they have completed a call with, store this information on a database, and warn the user appropriately depending on the data available.

The app was designed on Android Studio, an IDE (Integrated Development Environment) which provided all the required tools, including an emulator to allow the virtual testing of the app. It was programmed using Java and a GitHub repository was used to record code alterations and aid project collaboration. The Android Developer's and Firebase documentation as well as Stack Overflow was used to support the development of code.

Firstly, to allow the detection of incoming calls, the in-built Android telephony API (App Programming Interface) was used to communicate with the telephone service, allowing the app to determine the call state of the phone and identify incoming numbers. Code was used to instruct the app to compare these numbers to the user's contacts list and determine their presence or absence. The presence of this feature was essential to limit annoyance to the consumer sending unnecessary notifications for known numbers.

Following this, a Firebase real-time database was linked to the app to allow storage of data on numbers. Two categories were created in the database: 'trusted' and 'not trusted'.

To build up information in the database, a notification was created using the Android Developer's documentation to ask the user to classify a number they had completed a call with as trustworthy/not trustworthy. Depending on if the user presses 'trust' or 'report' the database would be updated. If the user presses 'trust' a value of +1 is added to that number's count in the 'trusted' section of the database. If the user presses 'report' a value of +1 is added to that number's count in the 'not trusted' section of the database. These two counts would later be used to calculate the trustworthiness of that number.

Depending on the number of overall classification of the unknown number in the database, the user would receive a notification showing the number of users that trust/don't trust the number. If there was no existing information on the number, the user would receive warning stating this.

Following the completion of the app's basic features, the prototype was presented to 5 classes of students who were surveyed on their opinions of the usability of the app. Overall, reviews were very positive. However, concerns regarding a lack of clarity of the warning notifications were raised. Following this, the notifications were modified to present users with a percentage of trustworthiness instead of raw values.

Following market research, several desirable features were identified to increase the appeal and functionality of the app to consumers. These included the presence of a call log of all known suspicious numbers, with an option of automatic call blocking. This automatic blocking feature could also apply to numbers that had passed a threshold number of negative reviews.

To prevent scammers bombarding their own numbers with positive ratings, a feature could be added so that each user would only be allowed to rate a specific number once.

Following testing and refinements of the app, the app was updated to work on Android 10, making it suitable for most modern Android devices. This involved additional code to change the system that detects the incoming number as the API previously used only functioned up to Android 8. Next the app was ported to a real device and after thorough testing all bugs that came up were fixed. Following this a call log section was added to the database which could be used in a UI for the app in the future. Multiple other features are still in development.

To conclude, after showcasing and testing our app 'ScamBlocker' we are happy that its features can protect users from the dangers involved with scam calls.

Meet the Team:

Fionn:

Hi, my name is Fionn Campbell. I'm 14 and I go to Moate Community School. I absolutely love science and technology, which is why I applied for BT Young Scientists. I have much experience in science and software - I have completed multiple CTYI courses at DCU, including astronomy, chemistry and experimental physics, which are subjects I especially enjoy. I also regularly attend CoderDojo Athlone, and have some minor experience in HTML, CSS, Python, JavaScript and Java. My dad is a software engineer that works with Java in the telecoms industry as well. As for other things about me, I love music and play the trumpet and guitar. I play rugby for the local club too. Scam calls are an incredibly frustrating issue that myself and everyone I know are impacted by. I am excited to see how our app could combat this problem and benefit the world.

Emma:

Hi, I'm Emma Keane. I am 13 and I go to Moate Community School. I love science and the theory of it. Unlike my partner Fionn, technology isn't my area of expertise, but that's what drew me to this BT Young Scientists Project is I love to be challenged and learning about new things. I'm very much looking forward to learning more about coding and programming. Outside of school I attend ballet, I love to read everything and anything and have particular interest in history, and most of all I love to organise. In my own house between all the phones we would get dozens of these scam calls a week, that is why I think that many people will benefit greatly from this app.



Project Aims and Objectives:

Aims:

Our project aims to design an app which will alert the user when they receive a fraudulent call and advise them on how to respond. This would help reduce the success rate of these scams and reduce the overall incidence of the issue if the app was used on a large scale.

Objectives:

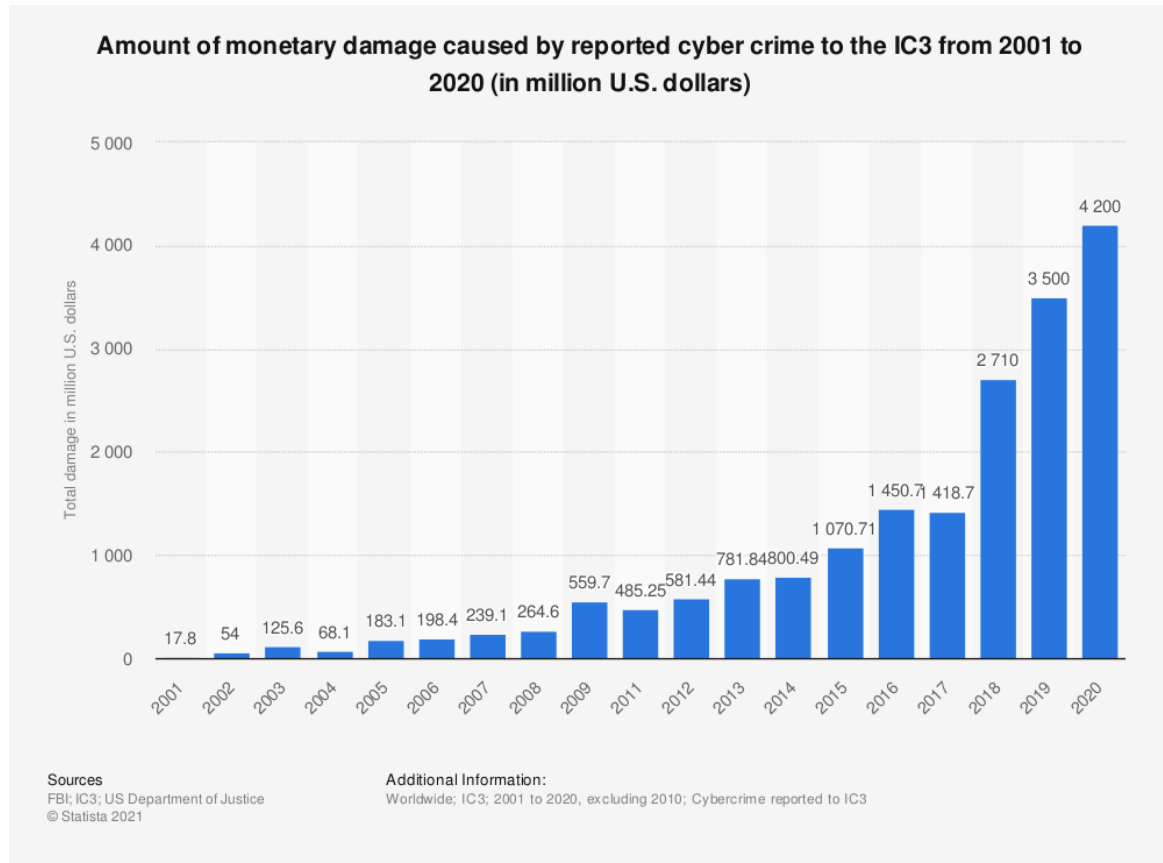
Conduct market research: survey students to determine multiple factors such as the scale of this issue locally, what audience the app should be targeted towards and specific features that users would like to be a part of the app.

Design app: using Android Studio, design app's basic features aimed for Android that will recognise numbers that are not on your contacts list, use user feedback to determine whether the number is trustworthy or not, store this information in a database and use it to warn users when that number calls them in the future. If possible, add desirable features such as an in-app UI, ability to block suspicious numbers automatically, app functionality on all versions of Android etc.

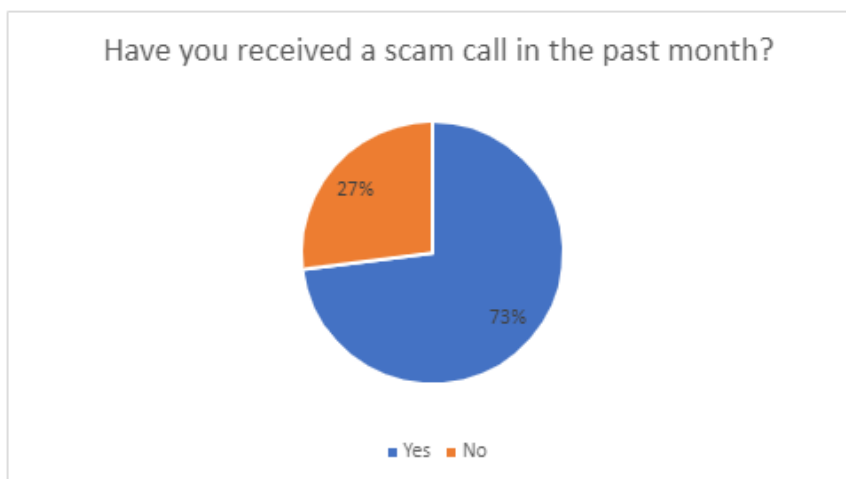
Showcase app and make improvements: showcase features to students and survey them on how effective they found the app's function to be and if they have any suggestions for adding/improving features. Improve the app based on this feedback.

Background Research:

Cybercrime is now commonplace in society. In 2007, Michel Cukier from the University of Maryland found that hackers were launching attacks on computers every 39 seconds. The 2020 Internet Crime Complaint Centre's report found that hackers were now launching attacks every 1.12 seconds, representing an almost 35 times increase.



In Ireland, scam calls are a massive issue that have surged 440% over the pandemic according to RTE. According to a survey by the EU earlier this year, Irish people are the second most likely citizens in the union to receive a scam call. A survey by thejournal.ie in July 2021 found 73% of the Irish people polled said they had received a call from an unknown Irish number in the past month.



Source: thejournal.ie

Many different methods of scamming exist which makes recognising the calls very difficult. These are the most common scams according to the Commission for Communication Regulation:

- Calling Line Identification Spoofing: when the number calling you, as displayed on your screen, has been faked by a scammer and appears to be a call from a genuine number. The scammer can then try to gain your trust and trick you into giving them personal info.
- SMS Spoofing: the number texting you has been faked by a scammer and appears to be a text from a real number. Like some of the other scams they are designed to gain your trust and encourage you to share information or trick you into clicking on a malicious attachment or link.
- 'Wangiri': this is the name given to short calls or faked missed calls that want you to call back an international number. Calling back will usually award the scammer money, often at your expense.
- Tech Support Scam Calls: calls where a scammer claims to offer a technical support service. The scammer will normally try to get you to give them remote access to your computer. If remote access is gained, the scammer can attempt to steal banking information, sensitive documents etc. In some cases, the scammer could hold your machine ransom until a payment is made to them.



Source: Help Net Security, June 2015

Even more scams exist, and they are ever adapting. People need to know that they are present and could perhaps ruin your life. This is not an exaggeration - scam victims are being conned out of an average of €5,300 with each scam with a total of €15.6m stolen according to figures from the Banking and Payments Federation Ireland in 2020.

Market Research Survey



Overview:

To help understand how people are affected by this problem locally, we created and distributed a short survey with some simple questions based on the issue to students in the school. Some of the questions would help reinforce the idea that this issue affects a large amount of people. Some more specific questions such as asking what service provider the respondent is with would highlight which networks are affected more. Other questions would give us an idea of what people think of the issue/app themselves, such as how they respond to scam calls and what functionalities they would like the app to have.

Distribution:

A link to the survey was sent out in an email on outlook to all students in the school. It was released in waves – 1st and 2nd years first, then 3rd years and TYs, and finally 5th and 6th years. Another final email was sent out later to remind people to complete the survey.

Survey questions:

1: What mobile network are you with?

☐ Vodafone

☐ Eir

☐ Three

☐ Virgin Mobile

☐ GoMo

☐ Other

2: Do you find that getting phone calls from suspicious-looking numbers is an issue?

☐ Yes

☐ No

3: How frequently do you receive phone calls from these suspicious numbers?

- ☐ Multiple times a day
- ☐ Daily
- ☐ Weekly
- ☐ Rarely
- ☐ Never

4: Do you often struggle to decide if you can trust these unknown calls?

- ☐ Yes
- ☐ No

5: What do you usually do when you get a call that you feel isn't trustworthy?

- ☐ Answer
- ☐ Decline the call
- ☐ Ignore it
- ☐ Block the number
- ☐ Report the number to the Gardaí
- ☐ Other

6: Did you know that scammers can 'spoof' (hide/change) their caller ID to cover up what they are doing?

- ☐ Yes
- ☐ No

7: Who do you think is most negatively impacted by these calls?

- ☐ Younger People (0-30)
- ☐ Middle-Aged People (30-65)
- ☐ Older People (65+)

8: Do you think you would benefit from an app that can detect scam calls?

☐ Yes

☐ No

9: How would you like the app to treat these calls when you receive one?

☐ Warn me about it

☐ Decline the call automatically

☐ Block the number permanently

☐ Report the number to the Gardaí

☐ Other

10: Would you like the app to keep you updated on the most popular scam numbers going around?

☐ Yes

☐ No

11: Would you like the app to notify you if it thinks your number has been stolen and is being used by a scammer?

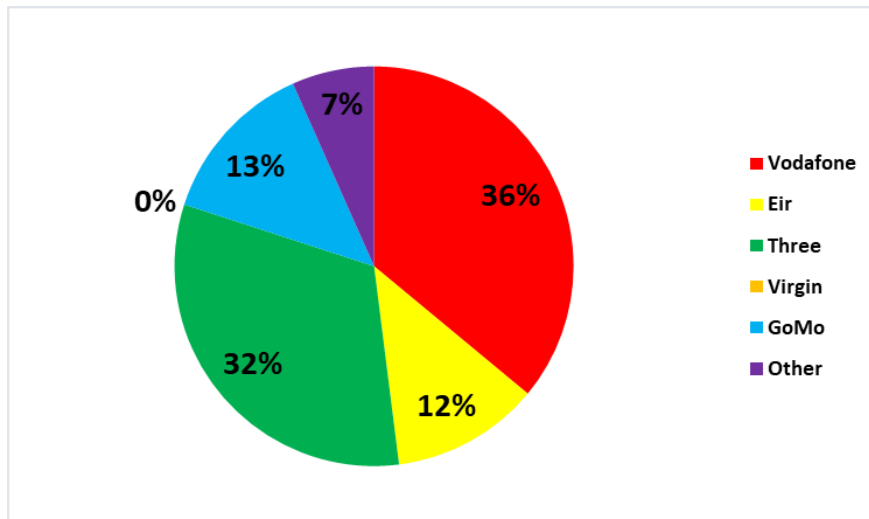
☐ Yes

☐ No

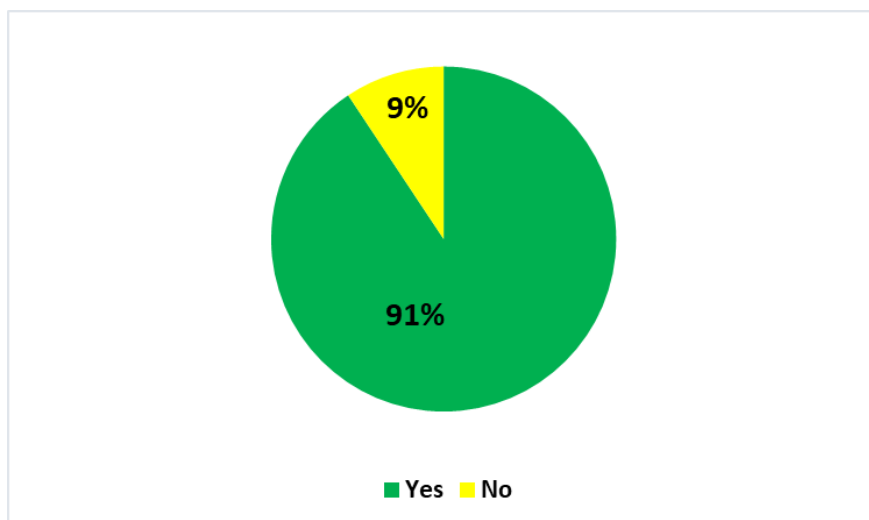
12: Any suggestions as to what else this app could do?

Survey Results:

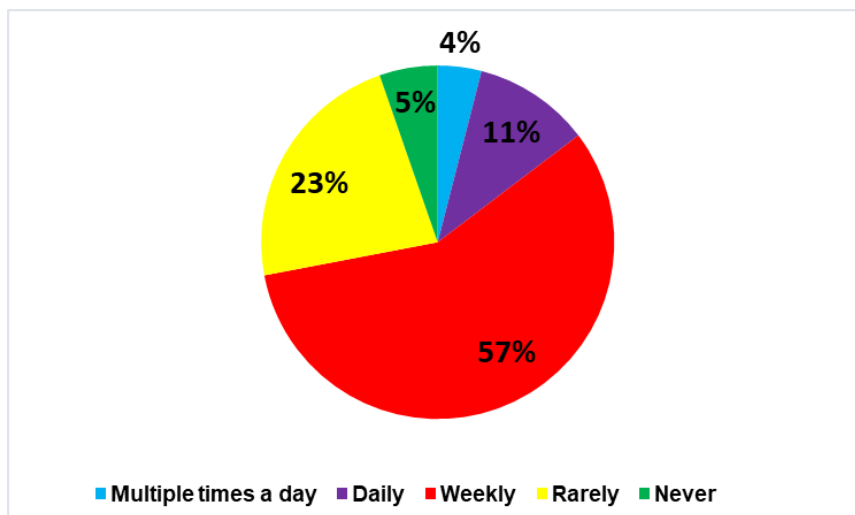
1: What mobile network are you with?



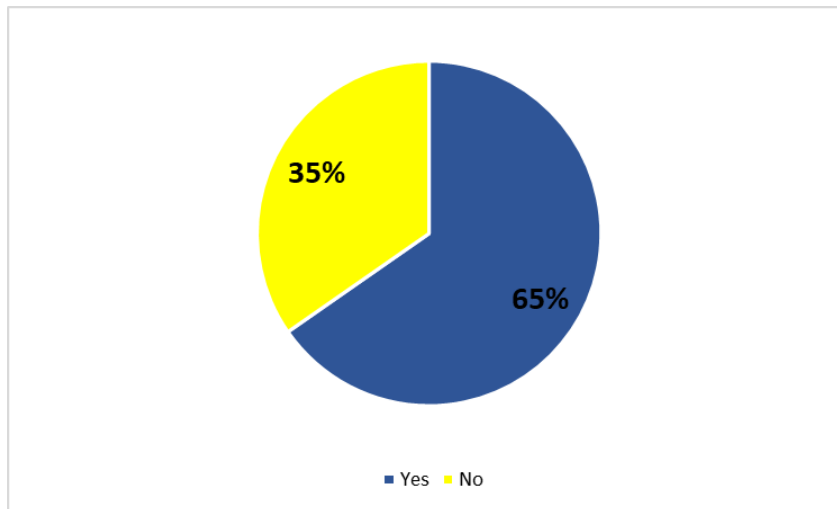
2: Do you find that getting phone calls from suspicious-looking numbers is an issue?



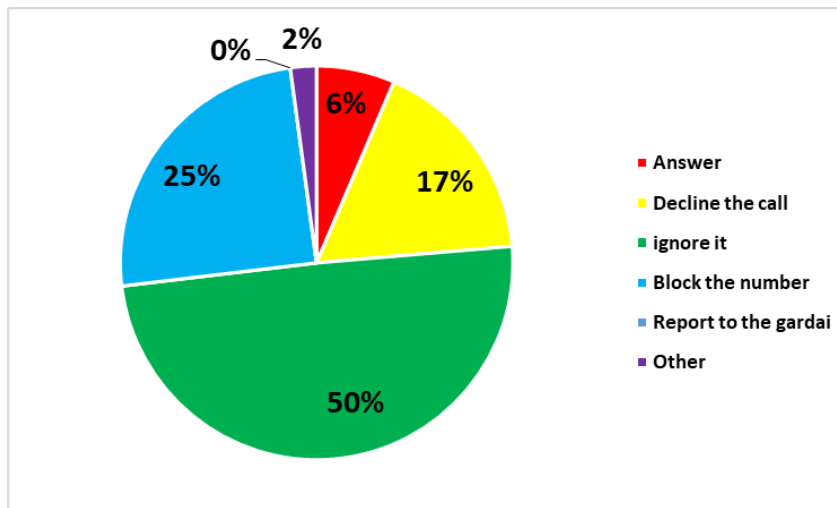
3: How frequently do you receive phone calls from these suspicious numbers?



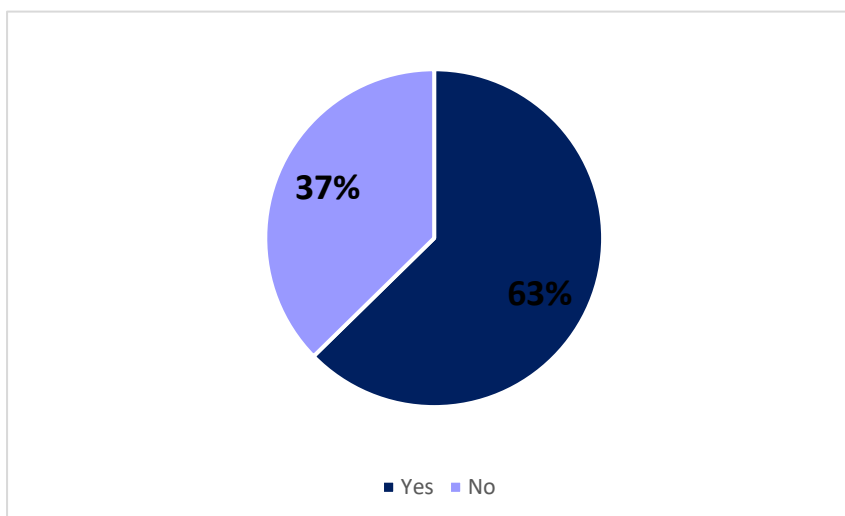
4: Do you often struggle to decide if you can trust these unknown calls?



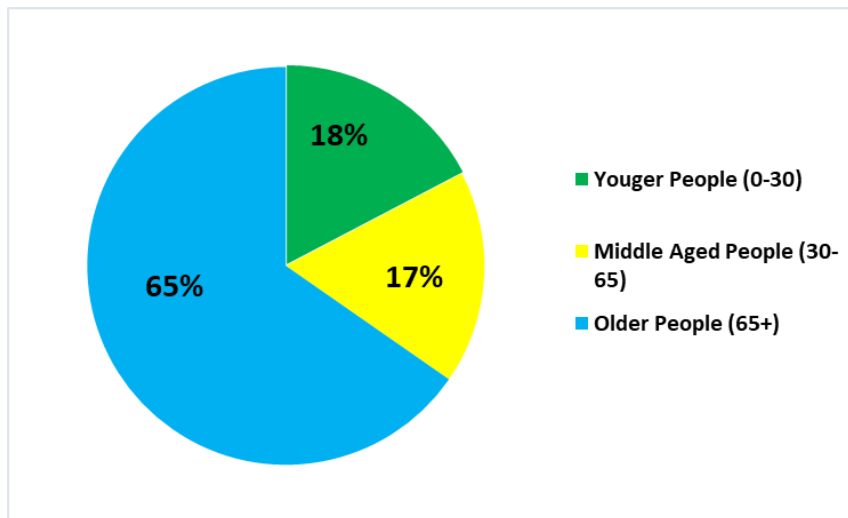
5: What do you usually do when you get a call that you feel isn't trustworthy?



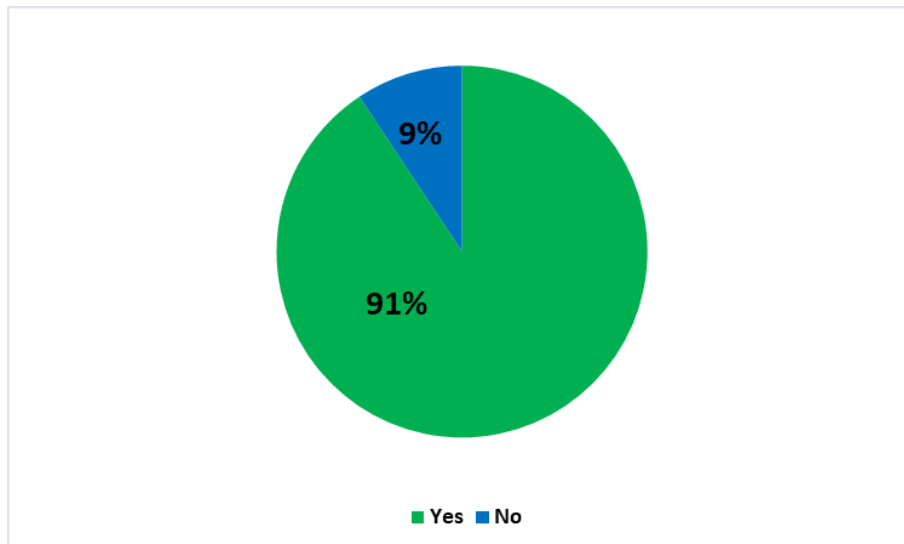
6: Did you know that scammers can 'spoof' (hide/change) their caller ID to cover up what they are doing?



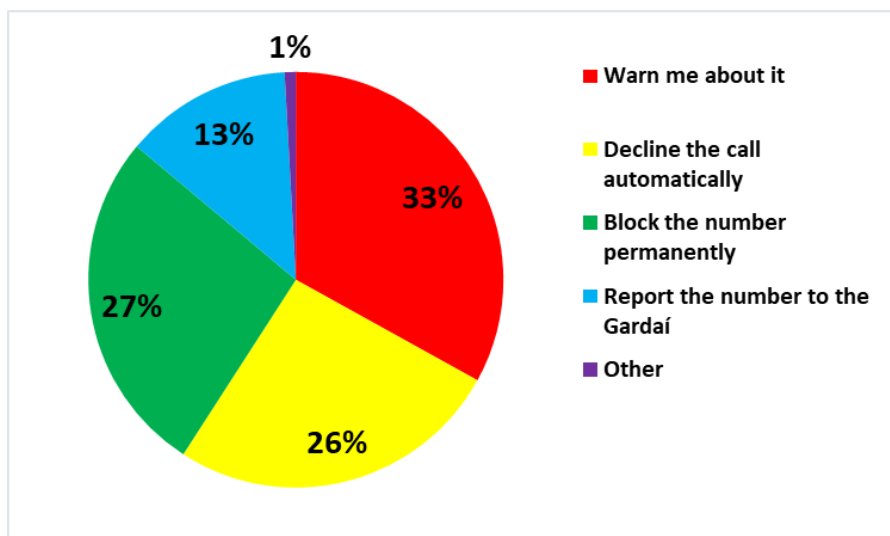
7: Who do you think is most negatively impacted by these calls?



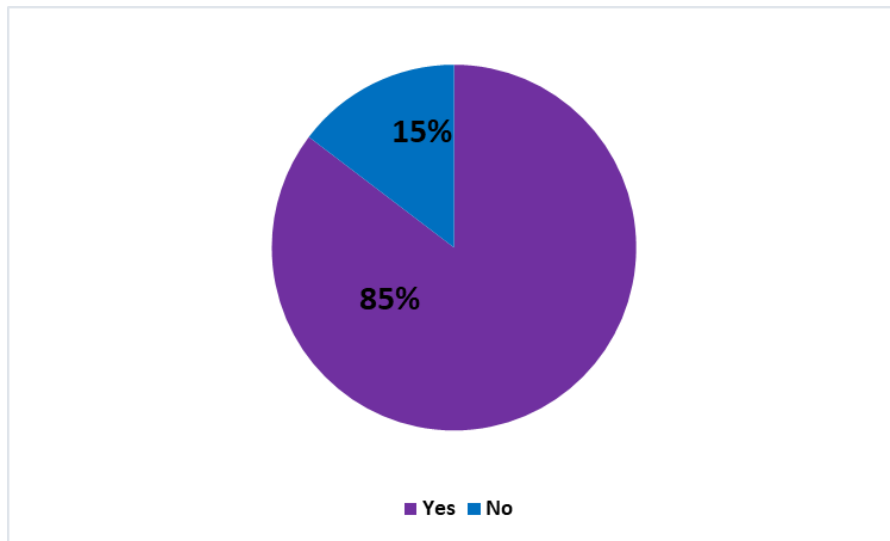
8: Do you think you would benefit from an app that can detect scam calls?



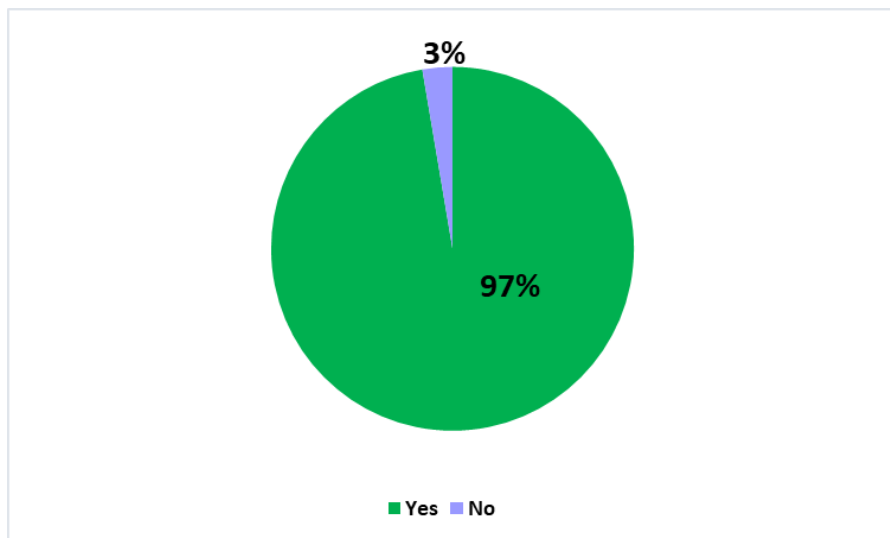
9: How would you like the app to treat these calls when you receive one?



10: Would you like the app to keep you updated on the most popular scam numbers going around?



11: Would you like the app to notify you if it thinks your number has been stolen and is being used by a scammer?



12: Any suggestions as to what else this app could do?

- Identify fake emails or texts as well.
- Tell what country the number is calling from.
- Try and find the location of the scammer.
- Give the number codes for different places.
- Let you know if the same scammer is ringing you with different numbers.

Conclusion:

Students receive calls from unknown numbers at least once a week. They find that this is an issue and that it is difficult to decide whether they can trust the caller. This mistrust in answering unknown numbers on your phone can lead to people missing valid calls, for example an appointment or a delivery. It is also a nuisance to receive these unwanted calls at inappropriate times such as while in class. All of this reinforces our motive for researching and investing in this project. There is a wide variety of users from different networks meaning that the issue is problematic everywhere. As we believed, most people think that the older generations are more vulnerable when it comes to answering and engaging with scam calls. Therefore, we would target our app especially towards this age group using this information. Almost every respondent agreed that our app would benefit them, a further reason to develop it.

Design Brief:

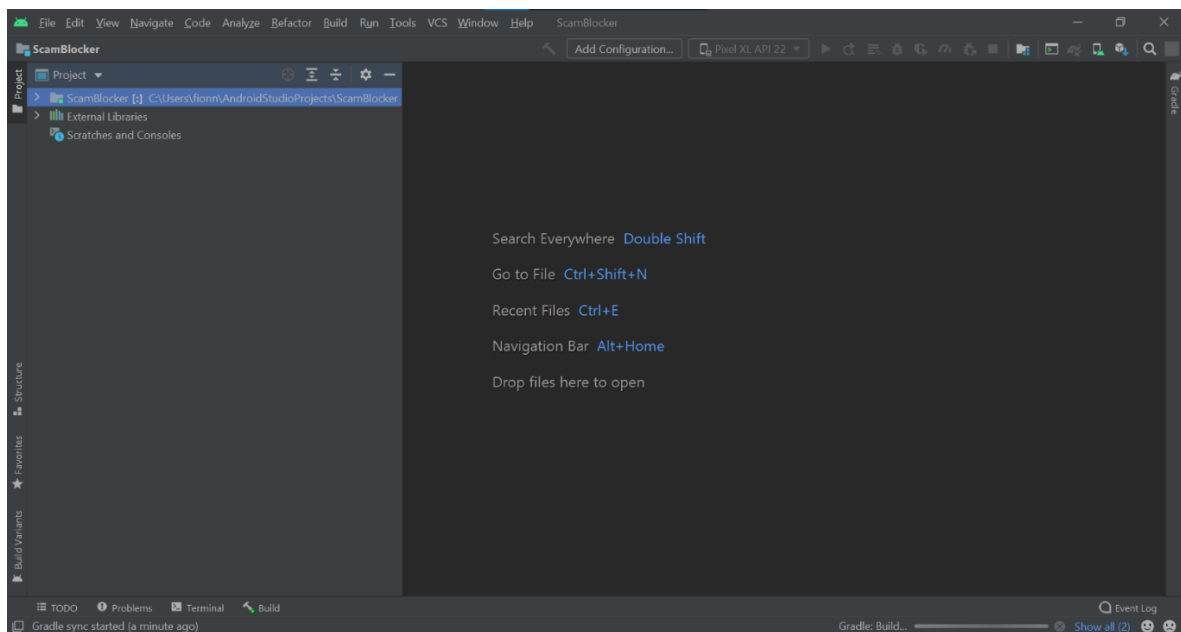
App Name	Scam Blocker
Operating Platform	Android
Target Audience	Android 8.0+ users
Timeline	September: Find out project theme, apply for BTYS, prepare survey. October: Send out survey, start coding app, set up database. Start prepping booklet, project diary etc. November: Continue and hopefully finish developing the basic app. Keep adding to booklet and diary. December: Polish app, add desirable features. Finish booklet and diary. January: Finish any documents that need to be written, make sure we understand all our material. Present project to judges.
Key App Features	1: Checking if a calling number is on the user's contacts list. 2: Asking users whether they trust an unknown number after they complete 3: Storing this feedback in a database. 4: Warning users if they are called by an unknown number with no info in the database. 5: Warning users appropriately if they are called by an unknown number that does have info in the database.
Desirable Features	Version of app working on a real phone. Automatic blocking of suspicious numbers. More informative warnings with colours, sounds etc. More advanced database security. Useful in-app UI. Recognising fraudulent text messages.

Experimental Methods:

Coding the app:

Setting up a Blank App:

Open Android Studio, go to the projects tab and press 'new project'. Follow these steps: Select a UI template: click on 'Phone and Tablet' and then 'empty activity'. The app prototype will not need an in-app UI. After that continue to the next screen and give the app a name: 'Scam Blocker'. Give the package name a unique name as this is what is used to identify all the app's resources. Select a language: Java, and a minimum SDK (the lowest version of Android the app will work on). This was set to Android 5 initially but had to be changed to Android 8 later (see '*creating high-priority notifications*'). Finish creating the project, which will show this screen. This is the environment in which the app will be designed, it contains all the required tools and resources to do this.



Manifest setup:

The manifest of the app is an XML (mark-up language like HTML) document that describes essential information to the Android OS, Android build tools and Google Play. This info includes the app logo, name, the permissions/services the app uses etc. Find it under project > app > src > main > androidmanifest.XML.

Most of the manifest can be left default, but some information does need to be added. (on next page). 'ServiceReceiver' is a class that will be set up later and will perform most of the features of the app. Add the below code beneath the <activity> block, which tells Android that this component exists and also describes an intent filter. An intent is like an action in Java. ServiceReceiver comes into action whenever the phone rings, hence it needing to know about the phone state so it can act accordingly.



```

<receiver
    android:name=".ServiceReceiver"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.PHONE_STATE" />
    </intent-filter>
</receiver>
<!-- Info about 'ServiceReceiver' class -->

```

As you may see, comments have been added to the code. Adding comments is a very good idea as it makes it easier to understand the code when looking back over it.

Outside the <application> block add the permissions the app needs to use. The notification permission as well as some of the other permissions may need to be enabled manually depending on the device. This is an issue you may encounter later.

```

<uses-permission android:name="android.permission.READ_CALL_LOG" />
<uses-permission android:name="android.permission.READ_PHONE_NUMBERS" />
<uses-permission android:name="android.permission.READ_CONTACTS" />
<uses-permission android:name="android.permission.READ_PHONE_STATE" />
<!-- Permissions the app uses -->

```

Creating the 'ServiceReceiver' class:

Create a new class called 'ServiceReceiver'. This is the class being used to perform most of the features of the app. Import the classes below, they will be used all throughout this class.

```

package com.bty.ssb;
// Package name

import android.Manifest;
import android.annotation.SuppressLint;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.database.Cursor;
import android.net.Uri;
import android.os.Build;
import android.provider.ContactsContract;
import android.telecom.TelecomManager;
import android.telephony.TelephonyManager;
import android.util.Log;

import androidx.annotation.NonNull;
import androidx.core.app.ActivityCompat;
import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;

```



```

import com.google.android.gms.tasks.OnSuccessListener;
import com.google.android.gms.tasks.Task;
import com.google.android.gms.tasks.TaskCompletionSource;
import com.google.android.gms.tasks.Tasks;
import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ServerValue;
import com.google.firebase.database.ValueEventListener;

import java.util.Date;
import java.util.HashMap;
import java.util.Map;
// Other classes imported for use in this class

```

Add this class declaration: 'ServiceReceiver' is an extension of the 'BroadcastReceiver' class, meaning it can do everything 'BroadcastReceiver' can on top of its own function. 'BroadcastReceiver' is a default Java class that receives broadcast events such as incoming calls from Android.

```

public class ServiceReceiver extends BroadcastReceiver {
    /* This class extends 'BroadcastReceiver', meaning it can do
    everything 'BroadcastReceiver' can as well as the features coded below */

```

Add these constants and variables. They will also be used throughout this class. It is easier to use these instead of writing their values in every place that they are needed.

```

private static String DB_URL =
    "https://scam-blocker-btys2021-default-rtdb.europe-west1.firebaseio.com";
private static final int NOTIFICATION_ID_VALUE = 123;
public static final String CHANNEL_ID = "CHANNEL_ID";
public static final String TRUST = "TRUST";
public static final String INCOMING_NUMBER = "INCOMING_NUMBER";
private static final String START_OF_CALL_ACTION = "START_OF_CALL_ACTION";
private static final String END_OF_CALL_TRUST_ACTION = "END_OF_CALL_TRUSTWORTHY_ACTION";
private static final String END_OF_CALL_SUSPICIOUS_ACTION = "END_OF_CALL_SUSPICIOUS_ACTION";
public static final String SUSPICIOUS_CALL_WEIGHTING = "suspiciousCallWeighting";
public static final String TRUSTWORTHY_CALL_WEIGHTING = "trustworthyCallWeighting";
public static final String NOTIFICATION_ID = "NOTIFICATION_ID";
/* Constants used in the code below, easier to use than strings in multiple places. It is
important that these are static as they are common across each instance of this class */

private static String previousState = TelephonyManager.EXTRA_STATE_IDLE;
private static Boolean incomingSuspiciousCallNotificationDismissed = Boolean.FALSE;
private static String lastIncomingNumber = "";
private static boolean callAnswered = false;
/* Variables maintaining important information that will be set and change as the app runs,
are also static and shared across each instance of 'ServiceReceiver' */

private final FirebaseDatabase db = FirebaseDatabase.getInstance(DB_URL);
private final DatabaseReference dbReference = db.getReference();
// Variables for API calls used to access Firebase, are constant.

```

Implementing the 'onReceive' method:

Add the code below (2nd part on next page). This method is an implementation of a method in the 'BroadcastReceiver' class, which is executed during a broadcast event. This implementation must be provided by the developer as it has no default function in its superclass (BroadcastReceiver). It will be executed multiple times throughout a phone call and will determine: the incoming number and phone call state, when to remove app notifications, when to start process for checking the incoming number in the contacts list/database, when the user has answered a call and when to start the process for working out whether to display a feedback notification. Make sure that the telephone permission is enabled on the phone (*under settings > apps and notifications > all apps > scamblocker > permissions*). The app won't work otherwise! This was an issue encountered during testing.

```
@Override
// Annotation that ensures the parameters in the below signature are not breached
public void onReceive(final Context context, final Intent intent) {
    /* Implementation of method in the 'BroadcastReceiver' class, which we must provide,
    it is called on multiple times by the app throughout a phone call */

    String state = "";
    // Variable to store current state of the phone, set later

    if (intent.getAction().equals(START_OF_CALL_ACTION)) {
        // If the user dismisses the start of call warning
        handleStartOfCallUserSelectedDismissOptionSelected(context, intent);
        // Initiate method to remove notification
    }
    else if (intent.getAction().equals(END_OF_CALL_TRUST_ACTION) ||
            intent.getAction().equals(END_OF_CALL_SUSPICIOUS_ACTION)) {
        // If the user selects an option on the feedback notification
        handleEndOfCallTrustOrReportOptionSelectedAction(context, intent);
        // Start method to report info to database and remove the notification
    }
    else {
        // During call initiation
        try {
            // Try to execute below code

            state = intent.getStringExtra(TelephonyManager.EXTRA_STATE);
            // Get phone call state and store it as variable
            String incomingNumber = intent.getStringExtra(TelephonyManager.EXTRA_INCOMING_NUMBER);
            // Get calling number and store it as variable

            if (state.equals(TelephonyManager.EXTRA_STATE_IDLE) &&
                    previousState.equals(TelephonyManager.EXTRA_STATE_OFFHOOK)) {
                // If the user has been called
                handleEndOfCallAction(context, incomingNumber);
                // Start method to work out whether to display feedback notification
            }

            if (previousState.equals(TelephonyManager.EXTRA_STATE_RINGING) &&
                    (state.equals(TelephonyManager.EXTRA_STATE_OFFHOOK))) {
                // If user answered call
                callAnswered = true;
                // Set variable to true for use in displaying feedback notification
            }
        }
    }
}
```



```

        if (state.equals(TelephonyManager.EXTRA_STATE_RINGING) &&
            previousState.equals(TelephonyManager.EXTRA_STATE_IDLE)) {
            // If there is an incoming call
            handleStartOfCallScamBlockChecking(context, intent, incomingNumber);
            // Start method for checking contacts and database and warning user accordingly
        }
    }
    catch (Exception e) {
        Log.e( tag: "Error", e.getMessage());
        // If error occurs during onReceive method execution print error message
    }
    finally {
        // Always executed at the end of a 'try' block
        previousState = state;
        // Save current state in above variable, used next time this method is executed
    }
}
}
}

```

Creating method to check user's contacts list:

Next, insert this method (2nd part on next page). This is what is being used to check the contacts list for an incoming number and is called by a different method (*handleStartOfCallScamBlockChecking: will be discussed later*) to determine whether to check the database. This was made with the help of an example on Stack Overflow (*see references*). Context is passed into this method for use in creating a cursor object and so is the incoming number for obvious reasons. Make sure that the app has access to the user's contacts (*under settings > apps and notifications > all apps > scamblocker > permissions*). This feature will not work without that permission.

```

public boolean isNumberInContactsList(Context context, String number) {
    // Method used to check contacts list for calling number
    boolean contactFound = false;
    // Sets variable to false by default
    if (number != null) {
        // Makes sure that the number calling has a proper value

        Uri lookupUri = Uri.withAppendedPath(ContactsContract.PhoneLookup.CONTENT_FILTER_URI,
            Uri.encode(number));
        /* Creates URI object that defines the search
        parameters: it tells the system what to look for */

        String[] phoneNumberProjection = {
            ContactsContract.PhoneLookup._ID,
            ContactsContract.PhoneLookup.NUMBER,
            ContactsContract.PhoneLookup.DISPLAY_NAME};
        // Defines a string array containing the fields to search within the contacts list

        try (Cursor cur = context.getContentResolver().query(lookupUri, phoneNumberProjection,
            selection: null, selectionArgs: null, sortOrder: null))
        /* Opens a cursor object which points to a result (the
        number, automatically closes when 'try' block completes */

```



```

try (Cursor cur = context.getContentResolver().query(lookupUri, phoneNumberProjection,
    selection: null, selectionArgs: null, sortOrder: null))
/* Opens a cursor object which points to a result (the
number, automatically closes when 'try' block completes */

{ if (cur.moveToFirst()) {
    // If contact exists in cursor object
    contactFound = true;
    // Sets this variable to true
}
}

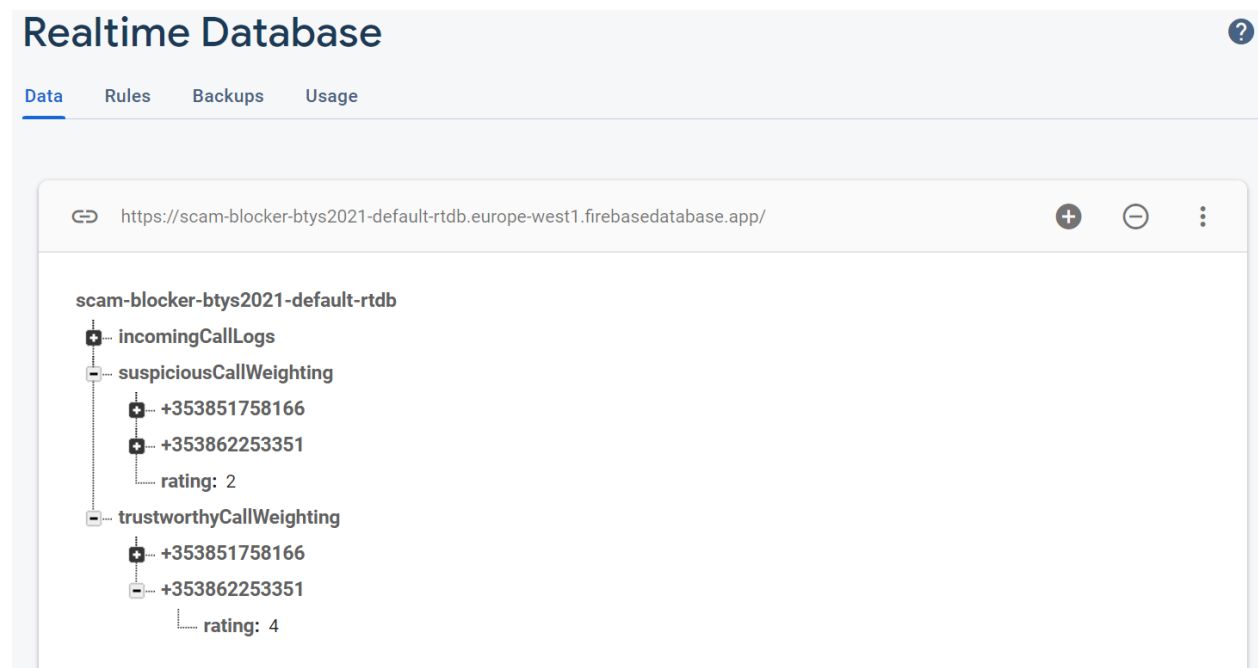
return contactFound;
/* Returns the contactFound variable - whether the
number is on the contacts list (true or false) */
}

```

Setting up the database:

Firebase is a flexible database which is compatible with Android and after research was deemed the appropriate platform for use in the context of this app. The real-time database feature was used which stores data in JSON format.

Go to the Firebase website and select 'Add project'. Give the project a name, which must be unique. Follow the steps and create the project. Add the app to the project by entering its package name (com.btys.sb). Navigate to the real-time database which will show this screen, a JSON tree containing all the data currently stored, as shown below.



There are two main important sections in this database: 'trustworthyCallWeighting' and 'suspiciousCallWeighting'. Later in this method a notification will be created which asks the user whether they want to trust or report an unknown number. This will be used to supply the aforementioned two sections with data. Any number that gets trusted by a user will have a count added to it in the 'trustworthyCallWeighting' section and any number that gets reported by a user will have a count added to it in the 'suspiciousCallWeighting' section. The two values across these sections should get bigger over time as more users rate the number and should be able to give an accurate representation of how trustworthy the number is. For example, the number above has had two negative reviews from users, and 4 positive reviews from users: 66% percent of users trust this number. This process of rating would be repeated for every new unknown number that users of the app interact with.

Next the required plugins and dependencies must be added to the app's Android build files to enable Firebase products. These can be found within the Firebase documentation (*see references*). Firstly, navigate to the project-level gradle build file (*under project > build.gradle*) and add the Google Services plugin to the dependencies block as shown below.

```
dependencies {  
    classpath "com.android.tools.build:gradle:7.0.3"  
  
    classpath 'com.google.gms:google-services:4.3.10'  
    // Google Services plugin that enables Firebase products  
}
```

Now open the app-level gradle build file (*under project > app > build.gradle*) and add this beneath the plugins block, which applies the Google Services plugin added above.

```
apply plugin: 'com.google.gms.google-services'  
// Enables Google Services plugin that enables Firebase products in this app
```

Finally go down to the dependencies block in the same file and add the below dependencies which are also required to enable the database features. The database URL is also required in the class that communicates with the database, but this was added earlier in the method as a constant.

```
implementation platform('com.google.firebase:firebase-bom:29.0.0')  
// Import the BoM for the Firebase platform  
implementation 'com.google.firebase:firebase-database:20.0.3'  
// Import library for the realtime database  
implementation 'com.google.firebase:firebase-firestore'  
// Import library for Cloud Firestore needed for the realtime database
```

Reading from and writing to the database:

Firstly, to write to the database, add these two methods. One of these will be called by a method that will be created later (*handleEndOfCallTrustOrReportOptionSelectedAction*) depending on if the user presses 'trust' or 'report' on a feedback notification which will also be created later. The code here was created following guidelines in the Firebase documentation (*see references*) and increments the number in question's value in the appropriate section of the database when executed. The incoming number is passed in as a parameter to the method to be looked up in the database. These two methods could be combined into one later to reduce the amount of code.

```
private void reportTrustworthy(String incomingNumber) {
    // Method for reporting trustworthy count to number's 'trusted' section in database
    Map<String, Object> updates = new HashMap<>();
    // Create a 'map', a key value pair which the database API call expects
    updates.put("trustworthyCallWeighting/" + incomingNumber + "/rating",
        ServerValue.increment(delta: 1));
    /* Defines key value pair as the address of the number's
    location in the database and what to update its value to */
    dbReference.updateChildren(updates);
    // Database API call to update its information
}

private void reportSuspicious(String incomingNumber) {
    // Method for reporting suspicious count to number's 'untrusted' section in database
    Map<String, Object> updates = new HashMap<>();
    // Create a 'map', a key value pair which the database API call expects
    updates.put("suspiciousCallWeighting/" + incomingNumber + "/rating",
        ServerValue.increment(delta: 1));
    /* Defines key value pair as the address of the number's
    location in the database and what to update its value to */
    dbReference.updateChildren(updates);
    // Database API call to update its information
}
```

Next insert this method (on next page), which is used to read from the database. It returns an asynchronous task (task not completed immediately) which when completed contains the desired value read from the database. This method is executed twice, once to get the trustworthy count and once to get the untrustworthy count, hence the type of count and the incoming number being passed in as parameters. The code here is again adapted from an example found in the Firebase documentation. A workaround had to be added to prevent an error in the context of this app. The 'onDataChange' method executes multiple times: when the listener is first attached to the number's address in the database and after every subsequent update at this address. The first execution of this gets the desired trusted/not trusted value associated with the calling number which is stored in the task object, marking its completion. If the database value changes again, which is likely at the end of a call during a feedback notification, 'onDataChange' will execute again, trying to complete a task that has already been completed. This results in a crash, an issue experienced whenever the feedback notification was interacted with until a workaround was implemented that would check whether the task was already completed, preventing the crash. Another way around this would be removing the listener after the database value has been read for the first time.



```

private Task<DataSnapshot> getCallWeightingTask(String type, String incomingNumber) {
    /* Method to read suspicious/trustworthy count from database. Returns a task object
    that will be executed asynchronously and later contain the desired value */

    final TaskCompletionSource<DataSnapshot> source = new TaskCompletionSource<>();
    // Variable for the task object to retrieve values

    DatabaseReference ref = db.getReference(type).child(incomingNumber + "/rating");
    /* API call to get a reference to the address in the
    database where the number and associated value is stored */

    ref.addValueEventListener(new ValueEventListener() {
        /* Adds a listener to this address which will detect
        the value already present as well as any updates to it */
    });
}

```

```

@Override
// Annotation that ensures the parameters in the below signature are not breached
public void onDataChange(@NonNull DataSnapshot dataSnapshot) {
    /* Executes upon an update from the listener: receives
    current value in database and subsequent updates to it */

    try {
        if (!source.getTask().isComplete()) {
            // VERY IMPORTANT makes sure that the task is not already complete
            source.setResult(dataSnapshot);
            /* Sets the result on the task object to the
            information from the database, completing the task */
        }
    }
    catch (IllegalArgumentException e) {
        // If an error occurs with the above method
        Log.e("tag: \"Error\", e.getMessage());
        // Print an error message to the debug terminal
    }
}
}

```

```

@Override
// Annotation that ensures the parameters in the below signature are not breached
public void onCancelled(@NonNull DatabaseError databaseError) {
    // If the read is cancelled for some reason
    Log.e("tag: \"Error\", msg: \"The read failed: \" + databaseError.getCode());
    // Logs an error message to the debug terminal
    source.setException(databaseError.toException());
    // Sets an exception (error) on the task object
}

});
return source.getTask();
// Returns the task object containing the result of the task
}

```

As the desired value is still stored in the task object, this method must be inserted which checks for a result and stores it as a string and then an integer. If the task's result is null, it will set this value to -1 which represents an empty value in that section of the database for that number. This method is called by another method (*handleStartOfCallScamBlockChecking*) which will be created later.

```
private int getWeightingFromTask(Task<DataSnapshot> task) {
    // Method for getting trusted/suspicious value from task that got it from database
    int weighting = 0;
    // Default integer weighting value to 0
    String weightingStr = null;
    // Default string weighting value to null
    if (task.getResult() != null) {
        // If task for database value returns an object
        if (task.getResult().getValue() != null)
            // If same task returned value
            weightingStr = task.getResult().getValue().toString();
        // Store returned value as a string variable
    }
    if (weightingStr != null) {
        weighting = Integer.parseInt(weightingStr);
        // If 'weightingStr' has a value turn it into an integer variable
    }
    else {
        weighting = -1;
        // If 'weightingStr' is null set variable to -1
    }
    return weighting;
    // Return the value: amount of positive/negative reviews a number has
}
```

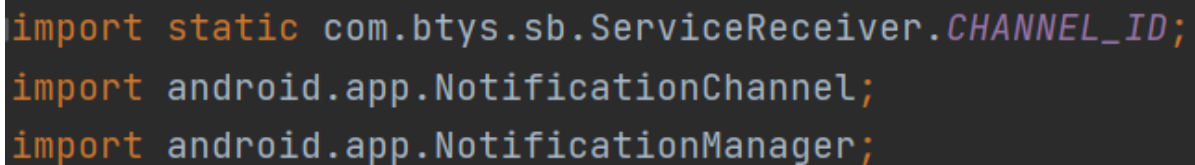
Creating high-priority notifications to warn users and ask for feedback:

Firstly, the minimum SDK must be changed as high-priority notifications only work on Android 8+. Change the below value to 26 (the API level of Android 8). It can be found in the app-level build.gradle file in the 'android' block.



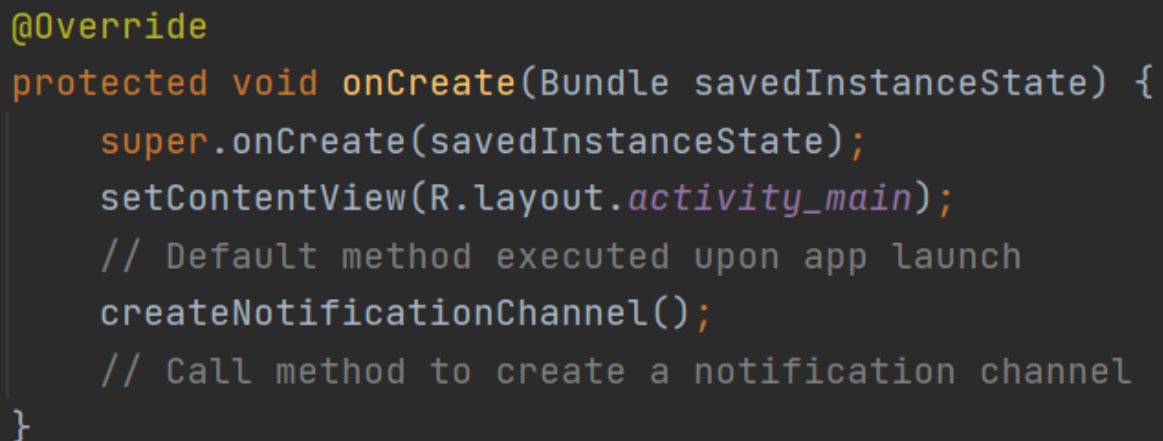
```
minSdk 26
```

Next go to the 'MainActivity' class and import these classes used in creating a notification channel, which is needed in displaying a notification according to the Android Developer's Documentation. The main structure of the code used to create the notification channel and notifications themselves can be found on this website (*see references*).



```
import static com.bty.ssb.ServiceReceiver.CHANNEL_ID;
import android.app.NotificationChannel;
import android.app.NotificationManager;
```

Call the method to create a notification channel in the 'onCreate' method which is executed upon launching the app.



```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    // Default method executed upon app launch
    createNotificationChannel();
    // Call method to create a notification channel
}
```

Create the method to create a notification channel for a high-priority notification. This code was written with the help of an example in the Android Developer's Documentation as aforementioned.

```
@RequiresApi(api = Build.VERSION_CODES.O)
// Hint to the compiler that this feature requires Android 8+
private void createNotificationChannel() {
    // Method to create notification channel

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        // If the device has the required Android version (8+)

        CharSequence name = "ScamBlocker";
        String description = "ScamBlockerChannel";
        int importance = NotificationManager.IMPORTANCE_HIGH;
        NotificationChannel channel = new NotificationChannel(CHANNEL_ID, name, importance);
        channel.setDescription(description);
        NotificationManager notificationManager = getSystemService(NotificationManager.class);
        notificationManager.createNotificationChannel(channel);
        /* Create a notifications channel which all of this app's notifications
        will pass through. Code sourced on the Android Developer's website */
    }
}
```

Next, add this method that calculates a percentage of users that trust/don't trust a number. It will be used in the warning notifications. The percentage is calculated as a floating-point number (number with decimals). Depending on which warning is being shown (whether the number is mostly trusted/not trusted), the trustworthy or suspicious count can be passed in as the numerator in the fraction in the percentage calculation to give the appropriate percentage of users that trust/don't trust the incoming number.

```
private float calculatePercentage(float trustCount, float suspiciousCount, float numerator) {
    return (numerator / (trustCount + suspiciousCount)) * 100;
    // Method for calculating percentage of users that trust/don't trust number
}
```

Now add these 3 methods (2nd part on next page) which are used to display different types of warning notifications when the user is called by an unknown number depending on the info on it in the database. Again, this code can be found on the Android Developer's website (*see references*). If a number has no info in the database, a warning stating that it is unknown will be shown, if it has mostly positive reviews, a notification stating that it is trusted will be shown, and if the reviews are mostly negative, a warning stating that the number is suspicious will be shown. Each notification has the option to dismiss it. Pressing on this option calls a method (*buildSuspiciousIntent*) that will be created later to build an intent object to go back to the 'onReceive' method and tell it to start the method to remove the notification. The three notifications here could be combined into a single method later to reduce the amount of code.

```
private void displayStartOfCallUnknownIncomingNumber(final Context context, String incomingNumber) {
    // Method to display number when there is no info on it in database

    PendingIntent suspiciousPendingIntent = buildSuspiciousIntent(context, incomingNumber,
        START_OF_CALL_ACTION);

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_launcher_foreground)
        .setContentTitle("Unknown number \" + incomingNumber + "\" ?")
        .setContentText("This number is not in your contacts list. Caution advised.")
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setFullScreenIntent(suspiciousPendingIntent, highPriority: true)
        .addAction(R.drawable.ic_launcher_foreground, title: "Dismiss",
            suspiciousPendingIntent);

    NotificationManagerCompat notificationManager = NotificationManagerCompat.from(context);
    notificationManager.notify(NOTIFICATION_ID_VALUE, builder.build());
    /* High priority notification builder with all relevant information
    included, sourced from the Android Developer's website */
}
```

```
private void displayStartOfCallWithWeighting(final Context context, String incomingNumber,
    Integer trustworthyCount, Integer suspiciousCount) {
    // Method for notification displayed when number is mostly trusted

    String message;
    // Variable for message displayed in notification, set later
    String endOfSentence = " of users trust this number.";
    // Variable for end of the message

    if (suspiciousCount == -1) {
        // If the suspicious count is null
        message = "100%" + endOfSentence;
        // Set the amount of users that do not trust the number to 100%
    }
    else {
        // If suspicious count has a positive integer value
        float percentage = calculatePercentage(trustworthyCount, suspiciousCount, trustworthyCount);
        // Calculate percentage of users who trust the number
        message = (int) percentage + "%" + endOfSentence;
        // Set message to percentage and end of sentence both defined above
    }
}
```



```

PendingIntent suspiciousPendingIntent = buildSuspiciousIntent(context, incomingNumber,
    START_OF_CALL_ACTION);

NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_ID)
    .setSmallIcon(R.drawable.ic_launcher_foreground)
    .setContentTitle("\"" + incomingNumber + "\"" + " is not in your contacts list")
    .setContentText(message)
    .setStyle(new NotificationCompat.BigTextStyle().bigText(message))
    .setPriority(NotificationCompat.PRIORITY_HIGH)
    .setFullScreenIntent(suspiciousPendingIntent, highPriority: true)
    .addAction(R.drawable.ic_launcher_foreground, title: "Dismiss", suspiciousPendingIntent);

NotificationManagerCompat notificationManager = NotificationManagerCompat.from(context);
notificationManager.notify(NOTIFICATION_ID_VALUE, builder.build());
/* High priority notification builder containing all relevant
    info. Sourced from Android Developer's website */
}

```

```

private void displayStartOfCallSuspiciousNotification(final Context context, String incomingNumber,
    Integer suspiciousCount, Integer trustworthyCount) {

    String message;
    // Variable for message to be set later
    String endOfSentence = " of users do not trust this number. Caution advised";
    // Second half of message

    if (trustworthyCount == -1) {
        // If the trustworthy count is null
        message = "100%" + endOfSentence;
        // Set the amount of users that do not trust the number to 100%
    }
    else {
        // If trustworthy count has a positive integer value
        float percentage = calculatePercentage(trustworthyCount, suspiciousCount, suspiciousCount);
        // Calculate percentage of users who do not trust the number
        message = (int) percentage + "%" + endOfSentence;
        // Set message to percentage and end of sentence both defined above
    }
}

```

```

PendingIntent suspiciousPendingIntent = buildSuspiciousIntent(context, incomingNumber,
    START_OF_CALL_ACTION);

NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_ID)
    .setSmallIcon(R.drawable.ic_launcher_foreground)
    .setContentTitle("Suspicious number \"" + incomingNumber + "\" ?")
    .setContentText(message)
    .setPriority(NotificationCompat.PRIORITY_HIGH)
    .setFullScreenIntent(suspiciousPendingIntent, highPriority: true)
    .addAction(R.drawable.ic_launcher_foreground, title: "Dismiss", suspiciousPendingIntent);

NotificationManagerCompat notificationManager = NotificationManagerCompat.from(context);
notificationManager.notify(NOTIFICATION_ID_VALUE, builder.build());
/* High priority notification builder with all relevant information
    included, sourced from the Android Developer's website */
}

```

Now, for the end of call feedback notification, add this method which displays the message asking the user whether they trust a number they have completed a call with. It gives them the option to 'trust' or 'report' it. These options, when selected, will call methods (*buildTrustedIntent* and *buildSuspiciousIntent*) that will later be created to generate intent objects to go back to the 'onReceive' method. Either a trusted/not trusted type intent is created which will tell 'onReceive' to start a function (*handleEndOfCallTrustOrReportOptionSelectedAction*) that will later be created to make the notification disappear and increment a trusted/not trusted count on the database.

```
void displayEndOfCallNotification(final Context context, String incomingNumber) {
    // Method for displaying end of call feedback notification

    PendingIntent trustPendingIntent = buildTrustedIntent(context, incomingNumber,
        END_OF_CALL_TRUST_ACTION);
    PendingIntent suspiciousPendingIntent = buildSuspiciousIntent(context, incomingNumber,
        END_OF_CALL_SUSPICIOUS_ACTION);

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_launcher_foreground)
        .setContentTitle("Received call from \"" + incomingNumber + "\" ")
        .setContentText("Do you trust this number?")
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .addAction(R.drawable.ic_launcher_foreground, title: "Trust", trustPendingIntent)
        .addAction(R.drawable.ic_launcher_foreground, title: "Report", suspiciousPendingIntent);

    NotificationManagerCompat notificationManager = NotificationManagerCompat.from(context);
    notificationManager.notify(NOTIFICATION_ID_VALUE, builder.build());
    /* High priority notification builder with all relevant information
    included, sourced from the Android Developer's website */
}
```

Adding behaviour to the notification buttons:

First, create these two methods that are called when the user interacts with the notifications previously created. These build intent objects that tell the 'onReceive' method what action the user has taken during a notification, so 'onReceive' can act accordingly. The trusted intent object tells 'onReceive' that the user has selected the trust option on the end of call notification. The suspicious intent object tells 'onReceive' that the user has selected the report option on the same notification but is also used to tell 'onReceive' that the user has dismissed the start of call notification. These are not confused as different information is passed to 'onReceive': whether the action occurred at the start or end of call.

```
private PendingIntent buildTrustedIntent(final Context context, final String incomingNumber,
                                       final String action) {
    /* Method to create intent with information that the user has selected the 'trust'
       option on the feedback notification to be passed back to the 'onReceive' method */

    Intent trustIntent = new Intent(context, ServiceReceiver.class);
    // Creation of intent and where it's being passed to
    trustIntent.setAction(action);
    // Sets intent to what kind of the action the user has taken

    trustIntent.putExtra(TRUST, value: true);
    trustIntent.putExtra(INCOMING_NUMBER, incomingNumber);
    trustIntent.putExtra(NOTIFICATION_ID, NOTIFICATION_ID_VALUE);
    // Further information to be included in the intent object

    return PendingIntent.getBroadcast(context, requestCode: 0, trustIntent,
                                     PendingIntent.FLAG_IMMUTABLE);
    /* Returns an intent that will be received by the
       'onReceive' method in this class with the required info */
}

private PendingIntent buildSuspiciousIntent(final Context context, String incomingNumber,
                                           String action) {
    /* Method to create intent with information that the user has selected the 'report'
       option on the notification to be passed back to the 'onReceive' method */

    Intent suspiciousIntent = new Intent(context, ServiceReceiver.class);
    // Creation of intent and where it's being passed to
    suspiciousIntent.setAction(action);
    // Sets intent to what kind of the action the user has taken

    suspiciousIntent.putExtra(TRUST, value: false);
    suspiciousIntent.putExtra(NOTIFICATION_ID, NOTIFICATION_ID_VALUE);
    // Further information to be included in the intent object

    return PendingIntent.getBroadcast(context, requestCode: 0, suspiciousIntent,
                                     PendingIntent.FLAG_IMMUTABLE);
    /* Returns an intent that will be received by the
       'onReceive' method in this class with the required info */
}
```

Add this method which will use code recommended by the Android documentation to remove a notification when it is called. This will be used later.

```
private static void cancelNotification(Context context, int notificationId) {
    // Method to make notifications disappear
    Object notificationManagerObj = context.getSystemService(Context.NOTIFICATION_SERVICE);
    NotificationManager notificationManager = (NotificationManager) notificationManagerObj;
    notificationManager.cancel(notificationId);
    // Code to make a notification disappear, sourced from Android Developer's website
}
```

Next, add this method, which is called by 'onReceive' when it receives an intent object stating that the user has pressed 'dismiss' on the notification. This calls the method (*cancelNotification*) to remove the notification.

```
private void handleStartOfCallUserSelectedDismissOptionSelected(Context context, Intent intent) {
    // If user presses 'dismiss' on notification
    int notificationId = intent.getIntExtra(NOTIFICATION_ID, NOTIFICATION_ID_VALUE);
    // Get notification ID from the intent object created
    cancelNotification(context, notificationId);
    // Start method to make notification disappear
}
```

Insert this method which will start the methods for removing the feedback notification (*cancelNotification*) and incrementing either section of the database (*reportTrustworthy* and *reportSuspicious*). This is called by 'onReceive' after it receives an intent object stating that the user has selected either the trust/report option on the notification.

```
void handleEndOfCallTrustOrReportOptionSelectedAction(final Context context, final Intent intent) {
    // When the user interacts with the feedback notification
    boolean reportSuspicious = false;
    // Defaults this variable used to know whether user reported the number to false.
    int notificationId = intent.getIntExtra(NOTIFICATION_ID, NOTIFICATION_ID_VALUE);
    // Gets feedback notification's ID, used to make it disappear
    String incomingNumber = lastIncomingNumber;
    // Create variable 'incomingNumber' and set value to the last number that called
    cancelNotification(context, notificationId);
    // Start method to make notification disappear

    if (intent.getAction().equals(END_OF_CALL_TRUST_ACTION)) {
        // If the user trusts number
        reportTrustworthy(incomingNumber);
        // Initiate system to report trustworthy value to the database
    }
    else {
        // If the user reports number
        reportSuspicious(incomingNumber);
        // Initiate system to report suspicious value to the database
    }
}
```

Displaying the appropriate warnings and feedback notification:

Insert this method (2nd part on next page) which firstly creates an asynchronous task to start the method that reads from the database. Once both these tasks are completed the 'onSuccess' method will execute, calling the method (*getWeightingFromTask*) to take the values from the completed database reading tasks. These values: the number of trusted/not trusted reviews a number has are then compared to decide which warning notification to display. Note that a thread sleep has been added after the tasks are completed. This is extremely important as without this workaround the warning notification will appear before the call notification causing the call notification to cover it. With the thread sleep the warning appears after the call notification does. Although not the most desirable fix, it was the best one found to solve this timing issue. This method is called by the 'onReceive' method whenever the phone rings.

```
private void handleStartOfCallScamBlockChecking(Context context,
                                                Intent intent, String incomingNumber) {

    // Executed when phone is ringing

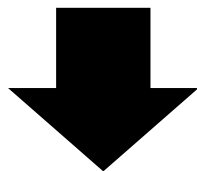
    boolean inContactsList = isNumberInContactsList(context, incomingNumber);
    // True or false variable that shows if the calling number is in your contacts list

    if (!(inContactsList)) {
        // If number is not in contacts list, executes below code

        final Task<DataSnapshot> suspicious = getCallWeightingTask(SUSPICIOUS_CALL_WEIGHTING,
                                                                    incomingNumber);
        final Task<DataSnapshot> trustworthy = getCallWeightingTask(TRUSTWORTHY_CALL_WEIGHTING,
                                                                    incomingNumber);
        // Request to fetch number's suspicious/trustworthy counts, isn't completed immediately

        final Task<Void> allTask = Tasks.whenAll(suspicious, trustworthy);
        // Two tasks above stored in one variable

        allTask.addOnSuccessListener(new OnSuccessListener<Void>() {
            // Adds listener to detect when both tasks are completed
```



```

public void onSuccess(Void aVoid) {
    /* Method executed when above tasks are both completed,
    decides which warning notification to display */
    try {
        int suspiciousCount = getWeightingFromTask(suspicious);
        int trustworthyCount = getWeightingFromTask(trustworthy);
        /* Sets these variables to the result of the method that gets the
        suspicious/trusted values from the task that gets them from the database */
        try {
            Thread.sleep( millis: 1000);
            /* VERY IMPORTANT prevents warning notification
            from displaying before the call notification */
        }
        catch (InterruptedException t) {
            Log.e( tag: "Error", t.getMessage());
            // Log message for debugging if sleep is interrupted
        }
        boolean suspect = (suspiciousCount > trustworthyCount) ||
            (trustworthyCount == -1 & suspiciousCount > 0);
        // Set 'suspect' variable to true if either above conditions are true
        boolean incomingNumberUnknown = (trustworthyCount == -1 & suspiciousCount == -1) ||
            (suspiciousCount == 0 && trustworthyCount == 0) ||
            (!(trustworthyCount > 0) && suspiciousCount == -1) ||
            (!(suspiciousCount > 0) && trustworthyCount == -1);
        // Set 'incomingNumberUnknown' variable to true if any of above conditions are true

        if (incomingNumberUnknown) {
            // If incoming number is unknown (no relevant data in database)
            displayStartOfCallUnknownIncomingNumber(context, incomingNumber);
            // Display notification for unknown suspicious number
        }
        else {
            if (suspect) {
                /* If incoming number is suspect (suspicious count
                in database greater than trustworthy count) */
                displayStartOfCallSuspiciousNotification(context, incomingNumber,
                    suspiciousCount, trustworthyCount);
                // Display notification for known suspicious number
            }
            else {
                /* If the trustworthy count in database
                is greater than the suspicious count */
                displayStartOfCallWithWeighting(context, incomingNumber,
                    trustworthyCount, suspiciousCount);
                // Display notification for trusted number
            }
        }
    }
}
}

```



```

        catch (Exception e) {
            Log.e( tag: "Error", e.getMessage());
            // Prints error message to debug screen if issue occurs with 'onSuccess' method
        }
    }
});

allTask.addOnFailureListener(e -> {
    Log.e( tag: "Error", e.getMessage());
    // If failure of tasks to get trusted/suspicious counts occurs print error message
});
}
}

```

Finally, add this method which will determine whether the feedback notification should be displayed. It is called by the 'onReceive' method at the end of a call and checks a variable set by the same method to see if the user answered the call. It will only display the notification if they have. This is important as it would be unfair to let users rate a number that they have not had any interaction with.

```

private void handleEndOfCallAction(final Context context, String incomingNumber) {
    lastIncomingNumber = incomingNumber;
    /* Set this static variable to the calling number for use later in
    method that does not receive the 'incoming number' local variable */

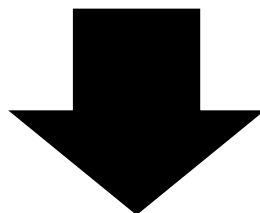
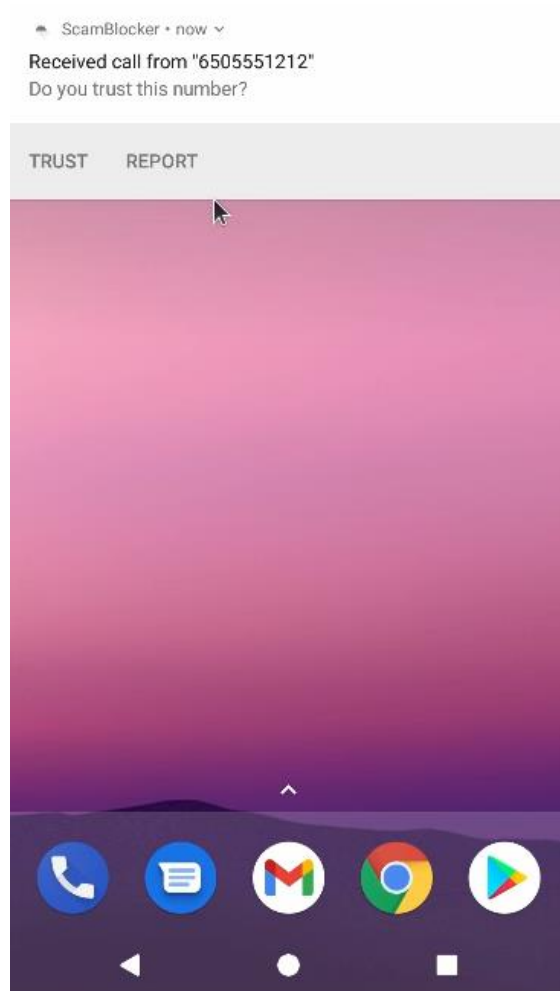
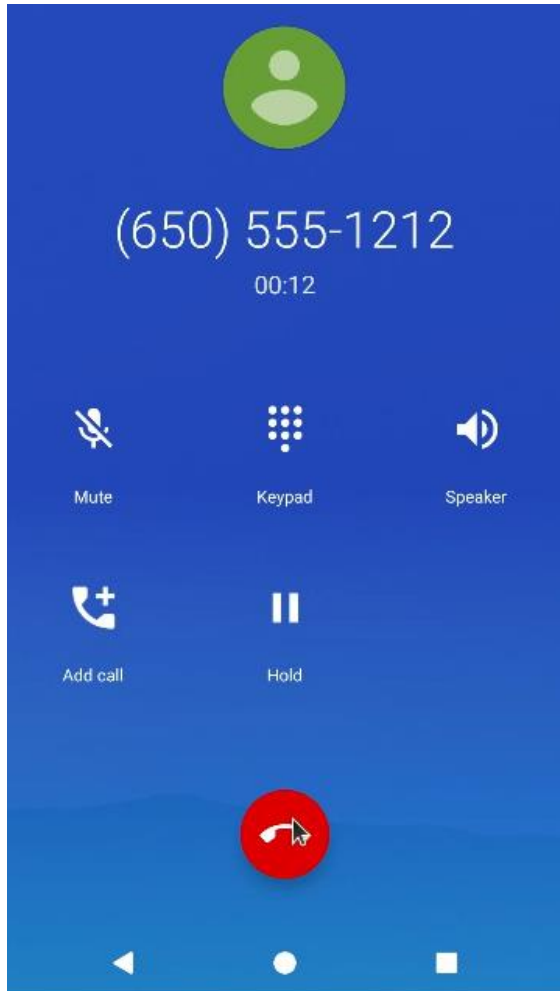
    if (callAnswered) {
        // If user answered the call
        displayEndOfCallNotification(context, incomingNumber);
        // Display feedback notification
        callAnswered = false;
        // Set variable back to false
    }
}
}

```

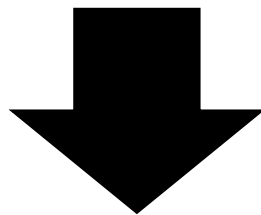
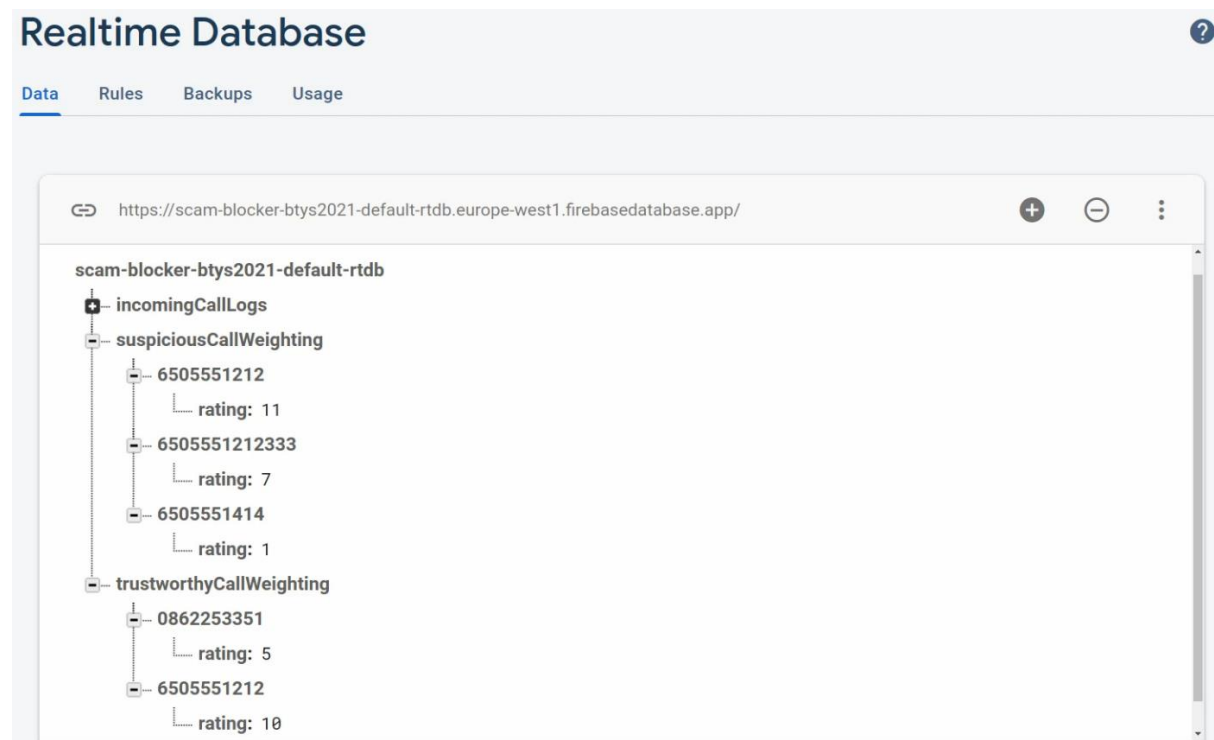
After thorough testing on an emulator, the above code gives a working prototype of this app for Android 8. Although, with most software, improvements can always be made. See 'Stand 2401 Appendices' (separate document) for methods on creating a version of the app which is tested and operational on a real Android 10 device.

App features in Action:

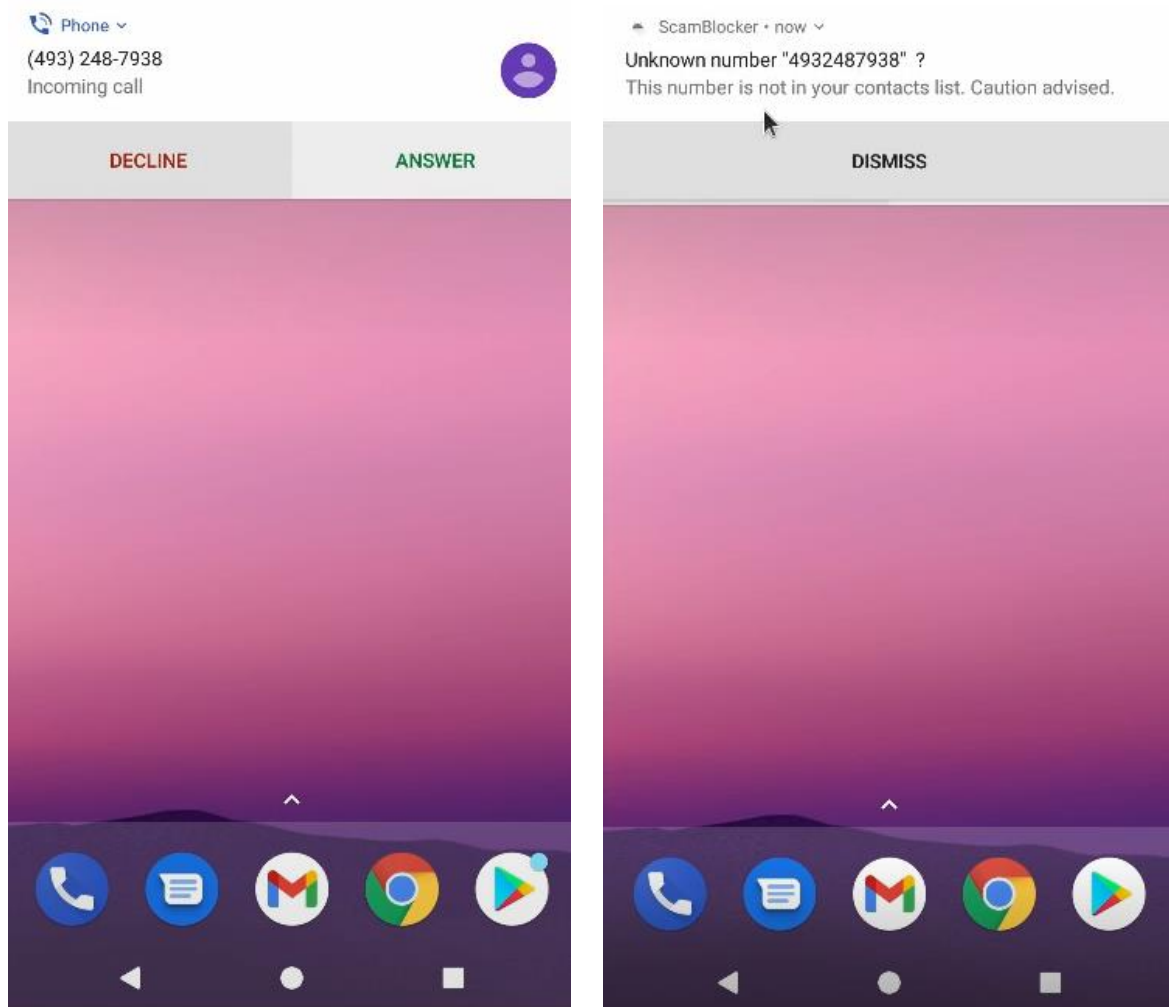
Notification to get user feedback on an unknown number that they have completed a call with: a count of +1 is added to the number in question's subsection in either the trusted/not trusted sections of the database shown below depending on whether the user presses 'trust' or 'report'



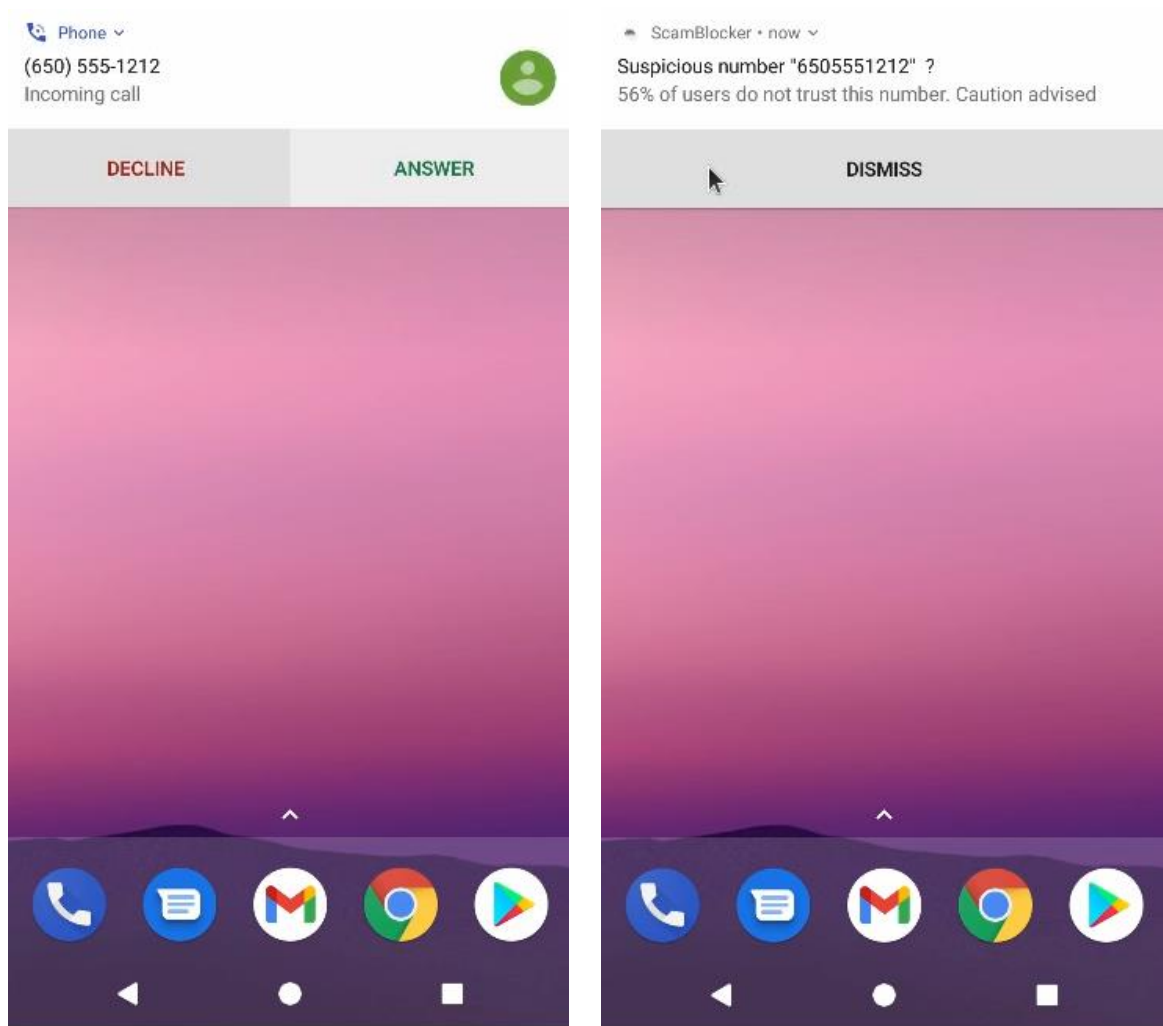
Database with information on numbers: the number of 'untrustworthy' reviews a user has given them stored in the 'suspiciousCallWeighting' section and the number of 'trustworthy' reviews a user has given them stored in the 'trustworthyCallWeighting' section. These values will be used to calculate a percentage of users that trust/don't trust the number and will be shown in a warning notification.



Warning shown when there is no information on an unknown number in the database: shown if there is no positive integer value attached to the calling number in either section of the database.



Warning shown when there is information on a calling number in the database: it will show a percentage of users that trust/don't trust the number depending on which integer value is higher in the database.



Project Conclusion:

Our app 'Scam Blocker' was successfully created with features that suit our consumer's needs. The market research survey carried out among 100 students was used to guide the app design process. The survey showed that there would be high demand for the product with 72% of participants stating they receive a suspicious call at least weekly and with 70% saying that they were unsure of what steps to take during a scenario like this.

The app which was coded using Java for Android has several key features. Firstly, it can detect when the phone rings and the incoming number. It does this using Android's in-built APIs that allow it to communicate with the telephone service. The app is linked to a database of existing numbers which it can use to warn users appropriately. To supply this database with information on numbers, if a call with an unknown number is answered and completed, the app sends a notification giving the user an opportunity to report the number as trusted/not trusted. Each number in the database has two integers attached to it across two sections, positive reviews and negative reviews. One of these values will be updated by 1 depending on if the user presses trust or report on a feedback notification.

Upon receiving a call from a number in the existing contacts list, the app takes no action. This was a key request from respondents to the initial market research survey to limit annoyance to the user. If the number is not in the contacts list and unlisted in the database, the user receives a basic warning to highlight a potential risk. If the number is in the database, the app will read the information present and warn the user appropriately depending on which number of reviews (positive/negative) is higher. For example, if a number has 15 negative reviews and 5 positive reviews the app will send a notification saying that the number is suspicious and that 15 users do not trust it and 5 users do trust it.

Following showcasing the app to students and surveying them for feedback, it was decided a percentage of users that trust/don't trust the number would make the warning clearer for the user. A version of the app that works on real Android phones was also requested. This was done by changing the system that detects incoming numbers to work on Android 10 as it only worked on versions up to Android 8 previously. A call log feature was also added to the database which could be used later in an app UI. This new version was also thoroughly tested on real devices to eliminate any bugs that may not apply to the simulated phone. There are still multiple other features being developed as of this date.

It is hoped that these features will adequately warn users about possibly dangerous numbers, making them think twice about putting themselves at risk, lowering the success rate of scam calls.

Recommendations:

An in-app UI could display a call log and give users an option to block suspicious numbers they have received permanently. It could also contain other features such as links to websites with advice on dealing with scams and hotline numbers providing help if you end up falling victim to a scam.

Adding code that makes it so that one number can only review another specific number once would prevent scammers from bombarding their own numbers with positive reviews, tricking our app into trusting them.

Requesting users to categorise an untrustworthy call as a tech support, postage, bank scam, etc. would allow more specific warnings.

Automatic declining of suspicious numbers, perhaps if the number has passed a certain threshold of negative reviews would make it easier for the user instead of them having to manually decline the call.

Adding a cloud function to calculate percentage of users that trust/don't trust a number, would be more efficient and desirable than crunching numbers in the app.

Adding a section to the database containing known trusted numbers such as those belonging to the government, businesses etc. may eliminate a lot of gathering data, making the app more accurate during its early stages.

A system that would notify users when the app thinks that their number has been stolen and is being used by a scammer would be very beneficial to victims of these scenarios. The app could predict this when a number's reviews are very mixed.

A system that carries out the same main features of the app but with the less common scam texts would be very beneficial to the user.

Adding working authentication to the database, as the default security created issues with reading/writing from it.

More feedback when you receive a warning such as colours shown or certain sounds played depending on how trusted the caller is may make it easier for users to decide if they themselves want to answer the call.

Changing the notifications used from high-priority to basic would theoretically allow the app to operate on versions of Android lower than 8.0 for users with older devices

Further code tidying would make the code easier to understand and possibly improve app performance.

Acknowledgements:

Many thanks to:

- Our teachers, Ms. Cusack and Ms. O'Sullivan who guided us along our journey with important feedback and advice
- Iain Campbell, Fionn's father who has much expertise in Java and provided guidance during the app design.
- Martina Curran from AIT who provided helpful feedback on the code and tested our understanding of it.
- Ms. Phelan our computers teacher who also provided useful feedback
- All students involved in our surveys.
- Lauren Fox, who gave extremely helpful feedback on the project video.
- Frank Rosney, Jack Nally, Mark Loddick, Jack Ryan, Ava Gahan, Sam Collis, Ciarán Morgan, Ciarán McCormack and Eoin Adamson who all provided much-needed emotional support

References:

Android Developers. 2021. *App Manifest Overview* | *Android Developers*. [online] Available at: <<https://developer.android.com/guide/topics/manifest/manifest-intro>> [Accessed 1 October 2021].

Android Developers. 2021. *CallScreeningService* | *Android Developers*. [online] Available at: <<https://developer.android.com/reference/android/telecom/CallScreeningService>> [Accessed 27 December 2021].

Android Developers. 2021. *Create a Notification* | *Android Developers*. [online] Available at: <<https://developer.android.com/training/notify-user/build-notification>> [Accessed 31 October 2021].

Android Developers. 2021. *Meet Android Studio* | *Android Developers*. [online] Available at: <<https://developer.android.com/studio/intro>> [Accessed 28 September 2021].

Android Developers. 2021. *Privacy changes in Android 10* | *Android Developers*. [online] Available at: <<https://developer.android.com/about/versions/10/privacy/changes#non-resettable-device-ids>> [Accessed 15 December 2021].

Android Developers. 2021. *Run apps on the Android Emulator* | *Android Developers*. [online] Available at: <<https://developer.android.com/studio/run/emulator>> [Accessed 28 September 2021].

Dwyer, S., 2021. *Vast majority of people have received a scam call from an Irish number in the past month*. [online] TheJournal.ie. Available at: <<https://www.thejournal.ie/scam-calls-irish-numbers-poll-5490391-Jul2021/>> [Accessed 28 October 2021]

Eng.umd.edu. 2021. *Study: Hackers Attack Every 39 Seconds*. [online] Available at: <<https://eng.umd.edu/news/story/study-hackers-attack-every-39-seconds>> [Accessed 28 October 2021].

Firebase. 2021. *Add Firebase to your Android project* | *Firebase Documentation*. [online] Available at: <https://firebase.google.com/docs/android/setup#next_steps> [Accessed 15 November 2021].

Firebase. 2021. *Authenticate with Firebase on Android using a Phone Number* | *Firebase Documentation*. [online] Available at: <<https://firebase.google.com/docs/auth/android/phone-auth>> [Accessed 31 December 2021].

Firebase. 2021. *Firebase Realtime Database* | *Firebase Documentation*. [online] Available at: <<https://firebase.google.com/docs/database/>> [Accessed 23 October 2021].

GeeksforGeeks. 2021. *How to Obtain the Phone Number of the Android Phone Programmatically?* - *GeeksforGeeks*. [online] Available at <<https://www.geeksforgeeks.org/how-to-obtain-the-phone-number-of-the-android-phone-programmatically/>> [Accessed 30 December 2021].

GitHub. 2021. *GitHub: Where the world builds software*. [online] Available at: <<https://github.com/>> [Accessed 29 September 2021].

Help Net Security. 2021. *86.2 million phone scam calls delivered each month in the U.S. - Help Net Security*. [online] Available at: <<https://www.helpnetsecurity.com/2015/06/17/862-million-phone-scams-calls-delivered-each-month-in-the-us/>> [Accessed 28 October 2021].

How to get phone number of an incoming call?, H., Abbasi, S., Shaikh, M. and Khetiya, C., 2021. *How to get phone number from an incoming call?*. [online] Stack Overflow. Available at: <<https://stackoverflow.com/questions/13154445/how-to-get-phone-number-from-an-incoming-call>> [Accessed 6 October 2021].

Ic3.gov. 2021. [online] Available at: <https://www.ic3.gov/Media/PDF/AnnualReport/2020_IC3Report.pdf> [Accessed 28 October 2021]

McGlynn, M., 2021. *Scam victims conned out of €5,300 on average as fraud increases*. [online] Irish Examiner. Available at: <<https://www.irishexaminer.com/news/arid-40714991.html#:~:text=A%20total%20of%20€15.6,of%2080%25%20reported%20in%202020.>>> [Accessed 28 October 2021].

McNulty, F., 2021. *Why scams have surged 440% during the pandemic*. [online] RTE.ie. Available at: <<https://www.rte.ie/news/primetime/2021/0921/1248137-scam-texts-phone-calls-pandemic/>> [Accessed 28 October 2021].

Region, F., Lima, A., Puffelen, F. and Larionov, A., 2021. *Firebase Realtime Database connection killed: Different Region*. [online] Stack Overflow. Available at: <<https://stackoverflow.com/questions/68806876/firebase-realtime-database-connection-killed-different-region>> [Accessed 18 November 2021].

Statista. 2021. *Cyber crime: reported damage to the IC3 2020 | Statista*. [online] Available at: <<https://www.statista.com/statistics/267132/total-damage-caused-by-by-cyber-crime-in-the-us/>> [Accessed 28 October 2021].

See 'Stand 2401 Appendices' for app usability survey overview, questions and results as well as methods for improving app.